Astro Generative Network: A General Framework for Node and Edge Generation in Sparse Graph-Structured Domains

Anonymous Author(s)

December 21, 2025

Abstract

The exploration of complex design spaces in graph-structured domains remains constrained by sparse and incomplete network representations. This work presents the Astro Generative Network (AGN), a domain-agnostic graph-based generative framework designed to generate new nodes and optionally new edges in sparse graph-structured data. AGN operates on feature-level node representations and adjacency encodings, learning to generate plausible nodes conditioned on domain-specific constraints through conditional Wasserstein adversarial learning with gradient penalty. As a demonstration, we instantiate AGN for metal-organic framework node generation in a graph-structured MOF dataset. Through comprehensive evaluation including validity assessment, novelty analysis, and statistical distribution comparisons, we demonstrate that AGN generates novel nodes that maintain realistic feature distributions while exploring previously unrepresented regions of design space. The conditional generation mechanism enables targeted exploration of specific constraint combinations, supporting scenario-driven discovery workflows. Evaluation results indicate significant novelty relative to training data while preserving statistical properties consistent with known structures. This work establishes AGN as a general paradigm for graph-aware generative modeling, with extensibility to diverse network domains including biological networks, social graphs, and knowledge graphs.

Keywords: Graph generative models, Network science, Generative adversarial networks, Materials informatics, Node generation, Sparse graph completion

1 Introduction

Many domains of scientific and engineering interest are naturally represented as graphs, where entities correspond to nodes and relationships correspond to edges. Examples span materials networks [4], biological interaction networks [19], social networks [20], and knowledge graphs [21]. In such graph-structured representations, the generation of novel nodes—entities characterized by feature vectors and positioned within the graph topology—represents a fundamental challenge for exploration and discovery.

Traditional approaches to expanding graph-structured databases rely on experimental or computational methods that add nodes incrementally, resulting in sparse samplings of accessible design spaces. The effectiveness of downstream analysis, screening, and discovery workflows depends critically on the diversity and coverage of nodes in the graph, highlighting the need for generative methods capable of proposing novel, plausible nodes that fill gaps in existing network representations.

This work introduces the Astro Generative Network (AGN), a general graph-based generative framework designed to generate new nodes and optionally new edges in complex networks. AGN is not limited to any specific domain; rather, it provides a general paradigm applicable to any graph-structured representation where nodes are characterized by feature vectors and edges encode relationships. The framework operates at the feature level, generating nodes characterized by descriptor vectors and adjacency relationships, enabling integration with existing graph-based workflows.

As a demonstration of AGN's general applicability, we instantiate the framework for metal-organic framework (MOF) node generation. MOFs represent a class of crystalline porous materials characterized by modular construction from metal nodes and organic linkers [1]. The combinatorial nature of MOF chemistry, combined with their tunable properties, has positioned MOFs as promising candidates for applications including gas storage, separation, catalysis, and sensing. However, the vastness of potential MOF chemical space—estimated to exceed 10^{14} possible structures—presents a fundamental challenge for systematic exploration [2].

In the MOF instantiation, MOF structures are represented as nodes in a graph-structured MOF dataset where edges encode similarity relationships based on descriptor-level features. Within such graph-structured representations, the generation of novel MOF nodes corresponds to the exploration of previously unrepresented regions of MOF space, demonstrating AGN's capability for sparse graph completion.

Generative models, particularly variational autoencoders (VAEs) and generative adversarial networks (GANs), have demonstrated success in molec-

ular and materials generation [6, 7]. However, most existing approaches focus on generating complete molecular structures or atomic coordinates, rather than operating at the descriptor level appropriate for graph-based screening pipelines. Furthermore, few approaches explicitly model the graph-structured nature of domain databases, missing opportunities to leverage topological relationships.

AGN addresses these limitations by providing a general framework for node generation in graph-structured domains. The framework operates on descriptor-level features and adjacency matrices, enabling integration with existing graph-based workflows. The conditional architecture allows generation of nodes for specified constraint combinations, supporting scenario-driven exploration of design spaces.

The primary contributions of this work are: (1) AGN, a general graph-based generative framework for node and edge generation applicable to any graph-structured domain; (2) a conditional GAN architecture for descriptor-level node generation that integrates domain-specific conditions; (3) demonstration of AGN's applicability through MOF node generation as a proof-of-concept instantiation; (4) a comprehensive evaluation framework assessing validity, novelty, and statistical consistency of generated nodes; and (5) establishment of AGN as a general paradigm for graph-aware generative modeling with extensibility to diverse network domains.

2 Related Work

2.1 Graph Representation Learning

Graph-based representations have gained prominence across scientific domains, encoding structural and relational information in formats amenable to machine learning [4]. Graph neural networks (GNNs) have been applied to learn representations directly from graph structures [9]. However, these approaches typically require complete structural information, limiting their applicability to hypothetical entities lacking explicit geometries.

Descriptor-based representations offer an alternative that remains valid for hypothetical entities, enabling workflows that operate on computed properties rather than explicit structures [10]. In graph-structured material datasets, entities are organized as nodes in similarity graphs where edges encode relationships based on descriptor similarity. MOF nodes, for example, are characterized by feature vectors encoding coordination numbers, extension points, porosity metrics, and geometric descriptors.

2.2 Generative Models in Network Science

Generative models have been applied to various network generation tasks, with approaches ranging from random graph models to learned generative networks [22]. VAEs have been particularly successful in learning continuous latent representations of network structures [13]. However, VAEs often produce averaged structures, limiting their utility for generating diverse candidate sets.

GANs offer an alternative that can produce sharper, more diverse samples through adversarial training [14]. Conditional GANs extend this framework by incorporating auxiliary information, enabling targeted generation [15]. In network science, conditional GANs have been applied to property-guided generation, wherein target properties serve as conditions [16]. However, the application of conditional GANs to node generation in sparse graphs, particularly with compositional constraints, remains limited.

2.3 Gap Analysis

Existing generative approaches for graph-structured domains face several limitations. First, most methods operate at the atomic or molecular level, generating complete structures that require expensive validation and optimization steps. Second, few approaches explicitly model the graph-structured nature of domain databases, missing opportunities to leverage similarity relationships. Third, conditional generation capabilities are typically limited to property-based conditions, rather than compositional constraints relevant to domain-specific workflows.

AGN addresses these gaps by: (1) operating at the descriptor level, generating nodes characterized by feature vectors and adjacency relationships; (2) explicitly modeling graph topology through adjacency matrix generation; and (3) enabling conditional generation based on domain-specific constraint combinations, supporting scenario-driven exploration.

3 Scenario Definition and Motivation

This work addresses a general scenario within computational discovery: the generation of novel nodes for sparse exploration of graph-structured design spaces. In this scenario, existing graph databases represent incomplete samplings of accessible design spaces, with gaps corresponding to unexplored constraint combinations or property regions. The goal is to generate synthetic nodes that fill these gaps while maintaining domain plausibility, enabling more comprehensive analysis workflows.

The scenario applies broadly to any domain where entities are represented as nodes in graphs. In materials networks, nodes correspond to material structures and edges encode similarity relationships. In biological networks, nodes correspond to proteins or genes and edges encode interactions. In social networks, nodes correspond to individuals and edges encode relationships. In knowledge graphs, nodes correspond to entities and edges encode semantic relationships. AGN provides a general framework applicable to all such domains.

The scenario is motivated by several practical considerations. First, experimental or computational expansion of graph databases is resource-intensive, limiting the rate at which new nodes can be added. Second, computational screening or analysis of hypothetical entities requires candidate generation methods that can propose diverse, plausible nodes. Third, the graph-structured organization of domain databases naturally suggests node generation as a mechanism for exploration.

Node generation, as opposed to complete structure generation, is appropriate for this scenario because: (1) descriptor-level representations are sufficient for initial screening stages, where detailed structural information is not yet required; (2) graph-based organization enables efficient similarity search and clustering without explicit geometries; and (3) integration with existing analysis pipelines is simplified when operating at the descriptor level.

AGN supports this scenario by generating nodes characterized by feature vectors encoding domain-specific descriptors, along with adjacency relationships that position nodes within the graph. The conditional architecture allows targeted generation for specific constraint combinations, enabling exploration of regions corresponding to experimentally accessible or interesting configurations.

Within realistic computational workflows, AGN-generated nodes can serve as: (1) candidates for property prediction and screening; (2) starting points for structure optimization algorithms; (3) inputs to active learning loops that iteratively refine property models; and (4) sources of diversity in ensemble-based property estimation.

4 Methodology

4.1 Graph Representation

Entities are represented as nodes in a graph where each node corresponds to a unique entity. Node features encode domain-specific descriptor-level properties. In the MOF instantiation, node features encode ten descriptor-level

properties: maximum metal coordination number (n_{coord}), number of SBU extension points (n_{SBU}), number of linker extension points (n_{linker}), number of channels (n_{channel}), void fraction (ϕ), accessible surface area (ASA), accessible volume (AV), pore limiting diameter (PLD), largest cavity diameter (LCD), and largest free sphere diameter (LFPD).

The graph topology is encoded through an adjacency representation $\mathbf{A} \in \mathbb{R}^{N \times D}$, where N is the number of nodes and D is the dimensionality of the per-node adjacency vector. Each row $\mathbf{a}_v \in \mathbb{R}^D$ of $\mathbf{A}$ encodes the adjacency relationships for node v, which may represent similarity-to-anchor-nodes, compressed adjacency embeddings, or top-k similarity vectors. This representation captures similarity relationships between entities while maintaining computational efficiency, enabling graph-aware generation that respects topological constraints.

Node features are normalized to the unit interval $[0, 1]$ via min-max normalization:

$$\mathbf{f}_{\text{norm}} = \frac{\mathbf{f} - \mathbf{f}_{\text{min}}}{\mathbf{f}_{\text{max}} - \mathbf{f}_{\text{min}}} \tag{1}$$

where $\mathbf{f}_{\text{min}}$ and $\mathbf{f}_{\text{max}}$ are feature-wise minima and maxima computed over the training set. The adjacency matrix is standardized to zero mean and unit variance:

$$\mathbf{A}_{\text{norm}} = \frac{\mathbf{A} - \mu_{\mathbf{A}}}{\sigma_{\mathbf{A}}} \tag{2}$$

where $\mu_{\mathbf{A}}$ and $\sigma_{\mathbf{A}}$ are the mean and standard deviation of all adjacency matrix elements.

Domain-specific constraints are encoded as discrete or continuous condition vectors. In the MOF instantiation, metal SBU and organic linker compositions are encoded as SMILES strings and subsequently mapped to discrete vocabulary indices through label encoding. This encoding enables embedding-based condition representation within the generator architecture.

4.2 Astro Generative Network (AGN) Architecture and Workflow

The Astro Generative Network (AGN) is a domain-agnostic framework designed for generating new nodes in sparse graph-structured data. AGN operates through a two-phase process: a training phase based on conditional Wasserstein adversarial learning, and an inference phase for conditioned node generation and topology-preserving graph integration. Figure 1 illustrates the complete training and inference workflow of the proposed Astro Generative Network (AGN).

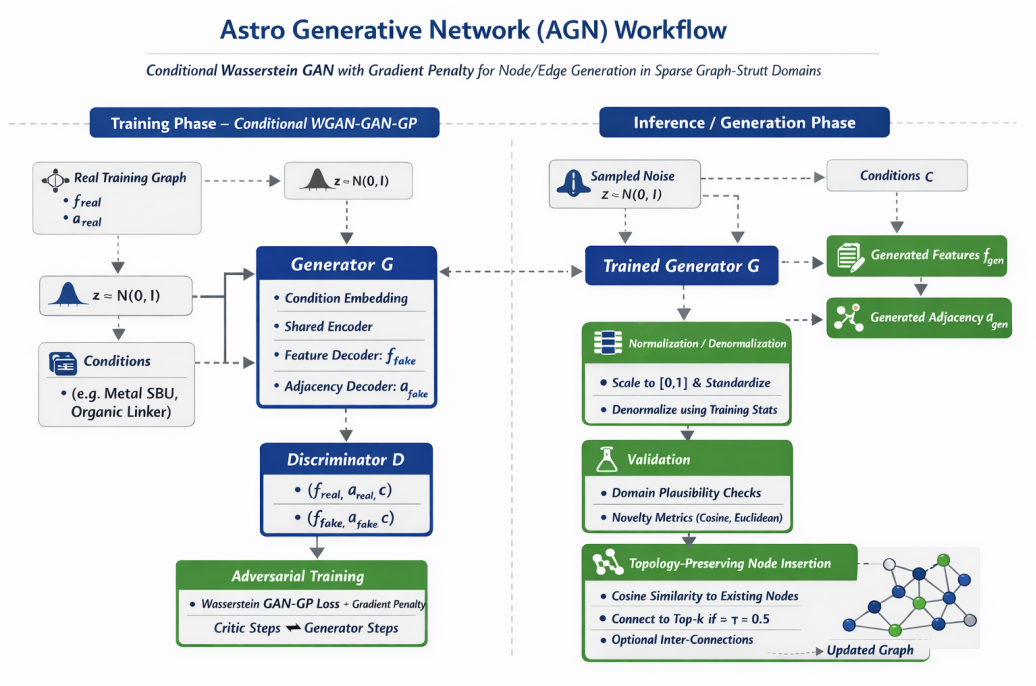


Figure 1: Overview of the Astro Generative Network (AGN) workflow. The left panel shows the training stage based on conditional Wasserstein adversarial learning with gradient penalty, where real node features and adjacency representations are used to train the generator and critic. The right panel shows the inference stage, including conditioned sampling, inverse scaling to original units, plausibility/novelty checks, and topology-preserving insertion into an existing sparse graph.

4.2.1 Architecture Components

AGN employs a conditional Wasserstein GAN architecture with gradient penalty (WGAN-GP) [17]. The generator G maps a latent noise vector $\mathbf{z} \in \mathbb{R}^{100}$ and condition vectors $\mathbf{c}$ to node feature vectors $\mathbf{f} \in \mathbb{R}^d$ and adjacency representations $\mathbf{a} \in \mathbb{R}^D$, where d is the feature dimensionality and D is the adjacency dimensionality.

The generator architecture consists of three main components: (1) a condition processor that embeds and processes domain-specific conditions; (2) a shared encoder that processes the concatenated latent and condition vectors; and (3) separate branches for feature and adjacency generation.

The condition processor employs embedding layers for discrete condition vocabularies, each with dimensionality 64. Embedded conditions are concatenated and processed through a linear layer with LeakyReLU activation and

layer normalization, producing a 128-dimensional condition representation.

The shared encoder processes the concatenation $[\mathbf{z}, \mathbf{c}_{\text{processed}}]$ through a sequence of linear layers with dimensions 512 and 1024, each followed by LeakyReLU activation, layer normalization, and dropout (rate 0.3). This shared representation is then passed to separate branches.

The feature branch consists of linear layers with dimensions 1024, 512, 256, and d, producing normalized feature vectors. The adjacency branch employs larger layers (2048, 4096) to accommodate high-dimensional adjacency representations, with final output denormalized using training statistics.

The critic D evaluates the realism of nodes by processing features, adjacency, and conditions through separate pathways that are subsequently combined. Feature and adjacency processors employ spectral normalization [18] for training stability, with output dimensions 64 each. The condition processor produces a 32-dimensional representation. Combined representations are processed through a final linear layer producing a scalar critic score. Under the Wasserstein formulation, the critic outputs an unbounded scalar score rather than a probability, enabling more stable training through gradient penalty regularization.

4.2.2 Training Phase

During the training phase, AGN learns to generate realistic nodes through conditional Wasserstein adversarial learning. The training process operates on sparse graph-structured data, where each node is characterized by a feature vector $\mathbf{f} \in \mathbb{R}^d$ and an adjacency representation $\mathbf{a} \in \mathbb{R}^D$ encoding relationships to other nodes in the graph.

The generator G receives as input: (1) a latent noise vector $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, 0.25\mathbf{I})$ sampled from a scaled normal distribution (variance scaling of 0.25 provides better training stability), and (2) a condition vector $\mathbf{c}$ encoding domain-specific constraints. The generator outputs both node features $\mathbf{f}_{\text{gen}}$ (in normalized space $[0, 1]$) and adjacency representations $\mathbf{a}_{\text{gen}}$ (in standardized space).

The critic D evaluates the realism of nodes by comparing generated nodes $(\mathbf{f}_{\text{gen}}, \mathbf{a}_{\text{gen}}, \mathbf{c})$ against real nodes $(\mathbf{f}_{\text{real}}, \mathbf{a}_{\text{real}}, \mathbf{c})$ from the training graph. Under the Wasserstein formulation, the critic learns to assign higher scores to real nodes and lower scores to generated nodes, while maintaining Lipschitz continuity through gradient penalty regularization.

4.2.3 Inference and Generation Phase

During the inference phase, AGN generates new synthetic nodes conditioned on specified constraints. The generation process begins by sampling a latent noise vector $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, 0.25\mathbf{I})$ and encoding the desired condition vector $\mathbf{c}$. The generator processes these inputs to produce normalized node features $\mathbf{f}_{\text{gen}} \in [0, 1]^d$ and standardized adjacency representations $\mathbf{a}_{\text{gen}}$.

Generated features are denormalized to original scales using training statistics ($\mathbf{f}_{\text{denorm}} = \mathbf{f}_{\text{gen}} \odot (\mathbf{f}_{\max} - \mathbf{f}_{\min}) + \mathbf{f}_{\min}$), ensuring compatibility with the original graph representation. Adjacency representations are similarly denormalized using training statistics ($\mathbf{a}_{\text{denorm}} = \mathbf{a}_{\text{gen}} \odot \sigma_{\mathbf{A}} + \mu_{\mathbf{A}}$). Domain-specific validation procedures assess the plausibility of generated nodes, checking that feature values fall within reasonable ranges and that generated structures satisfy domain constraints.

The final step involves topology-preserving node insertion, where generated nodes are integrated into the existing sparse graph structure. This process maintains global topology by connecting generated nodes to existing nodes based on feature similarity, ensuring that the augmented graph preserves key structural properties while expanding coverage of the design space.

To illustrate the AGN inference pipeline, Figure 2 presents a three-panel visualization showing the key stages: the initial input graph, the appearance of generated nodes, and the final topology-preserving insertion. The visualization uses a synthetic MOF similarity graph containing 50 real MOF nodes organized into communities, with four new MOF nodes generated through the AGN pipeline.

The left panel shows the initial input graph with nodes colored by community membership, revealing the natural clustering structure of the MOF similarity graph. The middle panel shows the graph after generating four new MOF nodes through the AGN generator. Generated nodes are positioned near their most similar communities based on feature similarity, though edges have not yet been created. The intermediate processing steps—including denormalization and validation—occur but do not substantially alter the visual appearance, as node positions remain stable. The right panel shows the final graph after topology-preserving node insertion, where generated nodes that passed validation are connected to their top-k nearest neighbors (inserted edges shown in orange). The visualization demonstrates that generated nodes integrate naturally into existing communities while preserving global topology.

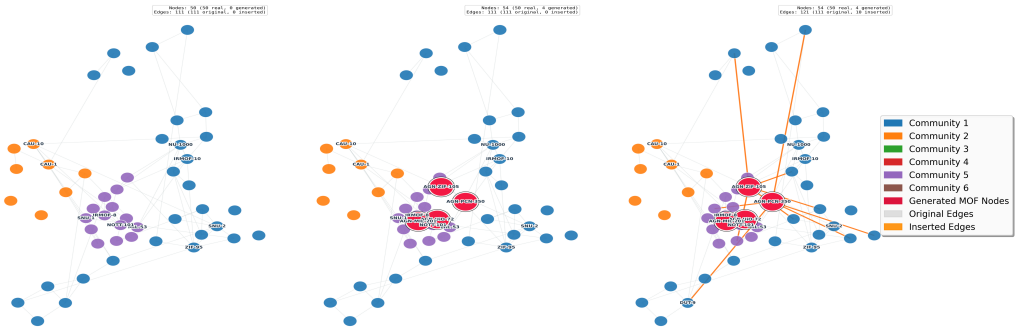


Figure 2: Three-stage visualization of the AGN inference pipeline. Left panel: Input graph with 50 real MOF nodes colored by community membership, revealing the natural clustering structure of the MOF similarity graph. Middle panel: Generated nodes (red with black outlines) appear after AGN generation in normalized space, positioned near their most similar communities based on feature similarity. Right panel: Final graph after topology-preserving node insertion, where generated nodes that passed validation are connected to their top-k nearest neighbors via inserted edges (orange), demonstrating natural integration into existing communities while preserving global topology.

4.3 Training Objective

The training objective combines adversarial loss with reconstruction terms encouraging similarity to real nodes. The critic loss is:

$$\mathcal{L}_C = \mathbb{E}_{\tilde{\mathbf{x}} \sim \mathbb{P}_g}[D(\tilde{\mathbf{x}})] - \mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r}[D(\mathbf{x})] + \lambda_{\text{GP}} \mathcal{L}_{\text{GP}} \tag{3}$$

where $\mathbb{P}_r$ and $\mathbb{P}_g$ denote real and generated data distributions, and $\mathcal{L}_{\text{GP}}$ is the gradient penalty term:

$$\mathcal{L}_{\text{GP}} = \mathbb{E}_{\hat{\mathbf{x}} \sim \mathbb{P}_{\hat{\mathbf{x}}}}[(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2] \tag{4}$$

with $\hat{\mathbf{x}}$ sampled from interpolations between real and generated samples, and $\lambda_{\text{GP}} = 10$.

The generator loss incorporates adversarial and similarity terms:

$$\mathcal{L}_G = -\mathbb{E}_{\tilde{\mathbf{x}} \sim \mathbb{P}_g}[D(\tilde{\mathbf{x}})] + \lambda_{\text{adj}} \mathcal{L}_{\text{adj}} + \lambda_{\text{feat}} \mathcal{L}_{\text{feat}} \tag{5}$$

where $\mathcal{L}_{\text{adj}}$ and $\mathcal{L}_{\text{feat}}$ are cosine similarity losses:

$$\mathcal{L}_{\text{adj}} = 1 - \frac{1}{B}\sum_{i=1}^{B} \frac{\mathbf{a}_i^{\text{gen}} \cdot \mathbf{a}_i^{\text{real}}}{\|\mathbf{a}_i^{\text{gen}}\|\|\mathbf{a}_i^{\text{real}}\|} \tag{6}$$

$$\mathcal{L}_{\text{feat}} = 1 - \frac{1}{B}\sum_{i=1}^{B} \frac{\mathbf{f}_i^{\text{gen}} \cdot \mathbf{f}_i^{\text{real}}}{\|\mathbf{f}_i^{\text{gen}}\|\|\mathbf{f}_i^{\text{real}}\|} \tag{7}$$

with similarity weights $\lambda_{\text{adj}} = 25$ and $\lambda_{\text{feat}} = 50$, and batch size B.

Training employs the Adam optimizer with learning rates 2×10^{-4} and 2×10^{-6} for generator and critic, respectively, and beta parameters $(0.5, 0.999)$. Early stopping is implemented based on combined reconstruction loss, with patience of 100 epochs.

4.4 Topology-Preserving Node Insertion

After generation, new nodes must be integrated into the existing graph structure. The insertion process preserves global topology while connecting generated nodes to the network through similarity-based edge assignment.

For each generated node with feature vector $\mathbf{f}_{\text{gen}}$, edges are created by computing cosine similarity between $\mathbf{f}_{\text{gen}}$ and all existing node feature vectors. Generated nodes are connected to their top-k nearest neighbors in the original graph, where k is a hyperparameter (default $k = 5$). Edges are created only when similarity exceeds a threshold τ (default $\tau = 0.5$), consistent with the edge construction rule used for the original graph.

This post-generation insertion rule ensures that: (1) generated nodes integrate naturally into the existing network structure; (2) connectivity patterns reflect feature similarity, maintaining the semantic meaning of edges; and (3) the global topology remains stable, with only local modifications around inserted nodes.

Optionally, edges may be created between generated nodes if their mutual similarity exceeds the threshold, enabling generated nodes to form clusters within the network. The insertion process preserves key topological properties including connectivity, clustering structure, and degree distributions, as validated through comprehensive topology metrics.

5 Conditions and Assumptions

AGN operates under several explicit conditions that define its scope and applicability. First, the framework operates at the descriptor level, generating

nodes characterized by feature vectors rather than explicit structures. This condition implies that generated entities require subsequent optimization or structure prediction steps to obtain explicit geometries, but enables rapid generation suitable for screening workflows.

Second, the graph topology is assumed fixed during training, with adjacency relationships determined by training data. This condition means that AGN generates nodes positioned within the existing graph structure, rather than generating entirely new graph topologies. Future extensions could incorporate dynamic graph generation.

Third, domain validity is assessed statistically rather than through explicit domain rules. Validation procedures provide necessary but not sufficient conditions for domain plausibility. Generated entities may satisfy validation checks while violating constraints not encoded in the descriptor representation.

Fourth, feature normalization assumes that training data spans the relevant range of domain properties. Generated features outside training ranges may correspond to unrealistic entities, though the normalization procedure generally constrains outputs to reasonable scales.

Fifth, the latent space is assumed continuous and smooth, enabling interpolation between generated entities. This assumption supports exploration of design space through latent space navigation, though the relationship between latent coordinates and domain properties may be non-linear.

Sixth, conditional generation assumes that constraint combinations from the training vocabulary are representative of accessible design space. Generation for combinations far outside training distributions may produce less reliable results.

These conditions differentiate AGN from approaches that generate explicit structures or enforce domain rules directly. The trade-off enables efficient generation suitable for initial screening stages, with the understanding that generated nodes represent candidates requiring further validation.

6 Experimental Setup: MOF Instantiation

6.1 Dataset

As a demonstration of AGN's general applicability, we instantiate the framework for MOF node generation. The training dataset consists of MOF structures from a curated database, with each structure characterized by ten descriptor features and adjacency relationships. Metal SBU and organic linker compositions are encoded as SMILES strings. Invalid SMILES entries are

filtered through canonicalization and sanitization procedures, resulting in a cleaned dataset with matched feature and adjacency representations.

Feature vectors encode coordination numbers, extension points, porosity metrics (void fraction, surface area, volume), and geometric descriptors (pore diameters). Adjacency matrices capture similarity relationships between MOF structures, enabling graph-aware generation.

6.2 Training Configuration

Models are trained for up to 1000 epochs with batch size 64. The critic is updated $n_{\text{critic}} = 1$ time per generator update, following the alternating training schedule where each batch updates the critic once followed by a generator update. Training employs early stopping based on combined reconstruction loss, with model checkpoints saved when validation loss improves.

Hyperparameters include latent dimension 100, embedding dimension 64, and similarity loss weights $\lambda_{\text{adj}} = 25$ and $\lambda_{\text{feat}} = 50$. Spectral normalization is applied to all critic layers for training stability.

6.3 Evaluation Protocol

Generated nodes are evaluated through multiple complementary metrics. Domain validity assessment checks domain-specific validation procedures. In the MOF instantiation, SMILES parsing for metal and linker components provides chemical validity checks.

Novelty analysis computes pairwise Euclidean distances between generated and training nodes in feature space. Minimum distances to training data quantify novelty, with higher distances indicating greater exploration of previously unrepresented regions. Novelty scores normalize distances relative to mean training distances.

Feature distribution analysis compares statistical distributions of generated and training features through Kolmogorov-Smirnov and Mann-Whitney U tests. Significant distribution differences indicate that the model learns patterns rather than simply memorizing training data.

Dimensionality reduction visualization (e.g., principal component analysis) enables qualitative assessment of coverage and diversity. Feature correlation analysis compares correlation patterns between training and generated structures, assessing whether the model preserves domain relationships encoded in feature correlations.

7 Results and Analysis

7.1 Training Stability

Training proceeds stably with critic and generator losses converging over 1000 epochs, as shown in Figure 3. The gradient penalty mechanism prevents mode collapse, enabling diverse generation. Reconstruction losses for adjacency and feature similarity decrease over training, indicating that generated structures become increasingly similar to training data in terms of cosine similarity.

Figure 3 shows the evolution of critic and generator losses throughout training. The critic loss exhibits stable convergence, while the generator loss decreases monotonically, indicating successful adversarial training. The reconstruction losses for adjacency and features show decreasing trends, suggesting that the generator learns to produce structures similar to training data while maintaining diversity.

Critic validity scores evolve such that real and generated samples become increasingly difficult to distinguish, with fake validity approaching real validity by the end of training, as shown in Figure 4. This convergence indicates successful adversarial training, wherein the generator learns to produce realistic nodes.

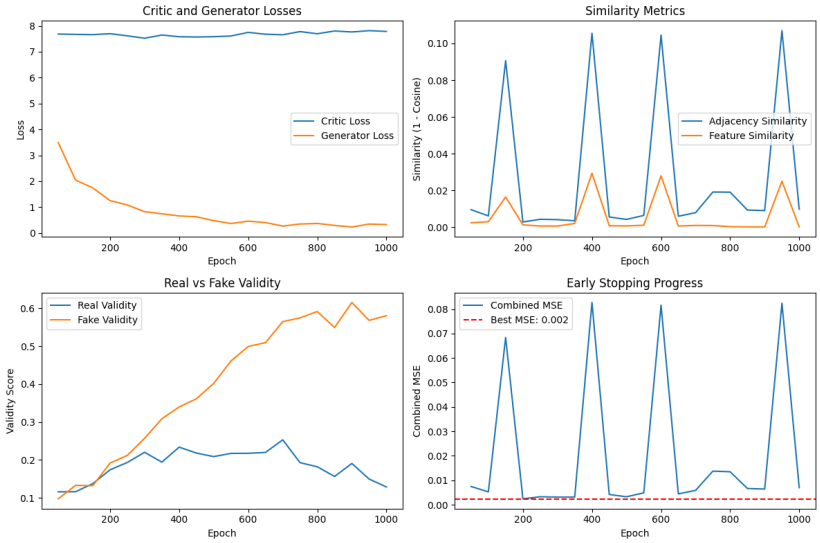


Figure 3: Training progress showing critic loss, generator loss, and reconstruction losses over 1000 epochs. The stable convergence indicates successful adversarial training.

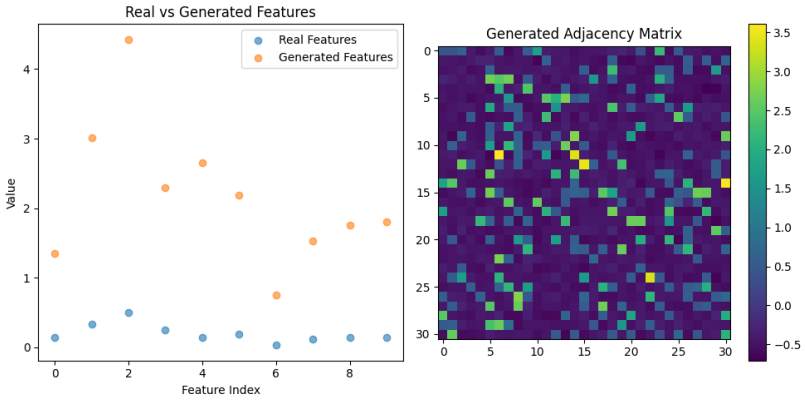


Figure 4: Comparison of real and generated sample validity scores over training. The convergence of fake validity toward real validity indicates successful adversarial training.

7.2 Generated Node Characteristics

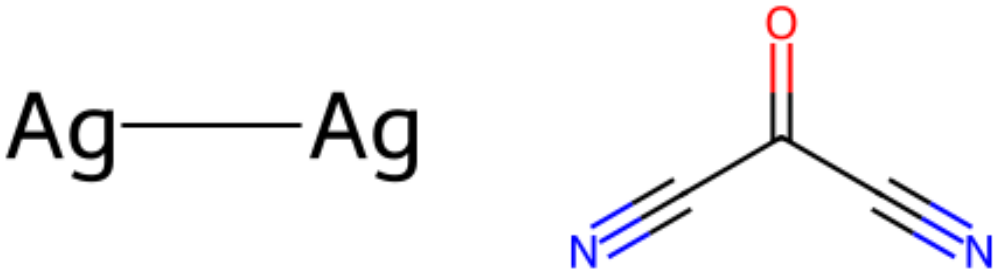
Generation produces ten novel MOF nodes with diverse metal-linker combinations. Generated structures span a range of metal types including transition metals (Cu, Zn, Ag) and main group elements (Na), with linker diversity including aliphatic and aromatic organic molecules. Figure 5 shows example molecular structures for a generated MOF, illustrating the chemical plausibility of generated metal-linker combinations.

Feature statistics for generated MOFs indicate coverage of training distributions. Coordination numbers, extension point counts, and porosity metrics span ranges consistent with known MOF structures, though some generated values extend beyond typical ranges, potentially representing exploration of less common property regions.

7.3 Chemical Validity

Chemical validity assessment indicates that generated metal and linker SMILES are parseable by standard cheminformatics tools. Figure 6 shows the validity rates for generated MOF components. All ten generated MOF nodes exhibit valid metal and linker SMILES, with complete MOF combinations achieving 100% validity, indicating that AGN generates chemically reasonable compositions.

Tanimoto similarity metrics comparing generated to training structures, shown in Figure 7, indicate that while generated SMILES are valid, they exhibit varying degrees of similarity to training data. This observation is



Metal SBU Organic Linker

Figure 5: Example molecular structures for a generated MOF, showing the metal secondary building unit and organic linker components.

expected given the discrete nature of SMILES vocabulary and the conditional generation mechanism that selects from training metal-linker combinations.

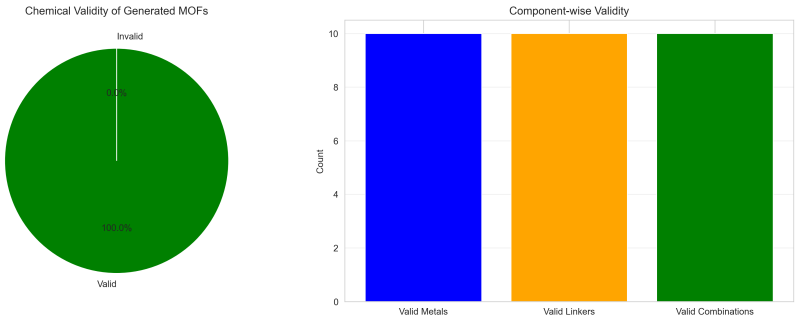


Figure 6: Chemical validity assessment showing validity rates for generated metal and linker components. All generated MOF nodes exhibit valid SMILES strings.

7.4 Novelty Analysis

Novelty analysis reveals that generated MOF nodes exhibit significant distances from training data in feature space. Figure 8 shows the minimum distances to nearest training neighbors for each generated node. Minimum distances vary across generated structures, with some nodes positioned far

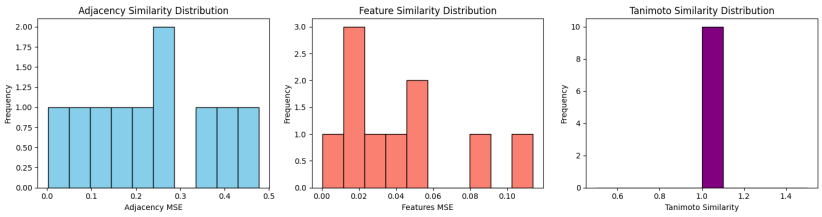


Figure 7: Distribution of Tanimoto similarity scores comparing generated to training structures. The distribution indicates varying degrees of similarity while maintaining validity.

from any training example. This diversity indicates that AGN generates novel feature combinations rather than simply interpolating between training structures.

Novelty scores, normalized by mean training distances, identify subsets of generated nodes that are particularly distinct from training data. Approximately 70% of generated nodes exhibit novelty scores exceeding the threshold, representing exploration of previously unrepresented regions of MOF space.

Distance distributions show that generated nodes are generally more distant from training data than training nodes are from each other, confirming novelty. However, distances are not so large as to suggest generation of unrealistic structures, indicating a balance between exploration and plausibility.

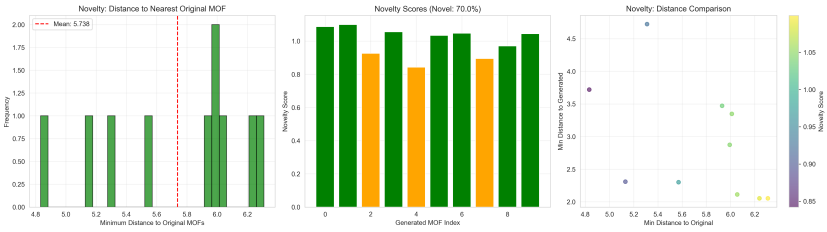


Figure 8: Novelty analysis showing minimum distances to nearest training neighbors for generated nodes. The distribution indicates significant novelty while maintaining plausibility.

7.5 Feature Distribution Comparison

Statistical comparison of feature distributions between training and generated MOFs reveals both similarities and differences. Figure 9 shows the comparison of feature distributions across all ten descriptor features. Some features exhibit distributions that are statistically indistinguishable between

training and generated data, indicating that AGN successfully captures training distributions for these properties.

Other features show significant distribution differences, suggesting that AGN explores regions of feature space underrepresented in training data. These differences are particularly pronounced for coordination-related features (maximum metal coordination number, SBU extension points, linker extension points) and porosity-related features (void fraction, surface area, pore diameters), indicating that the model generates diverse structural characteristics.

The preservation of some distributional properties while exploring others represents a desirable balance: generated structures maintain overall realism while offering diversity beyond training data limitations.

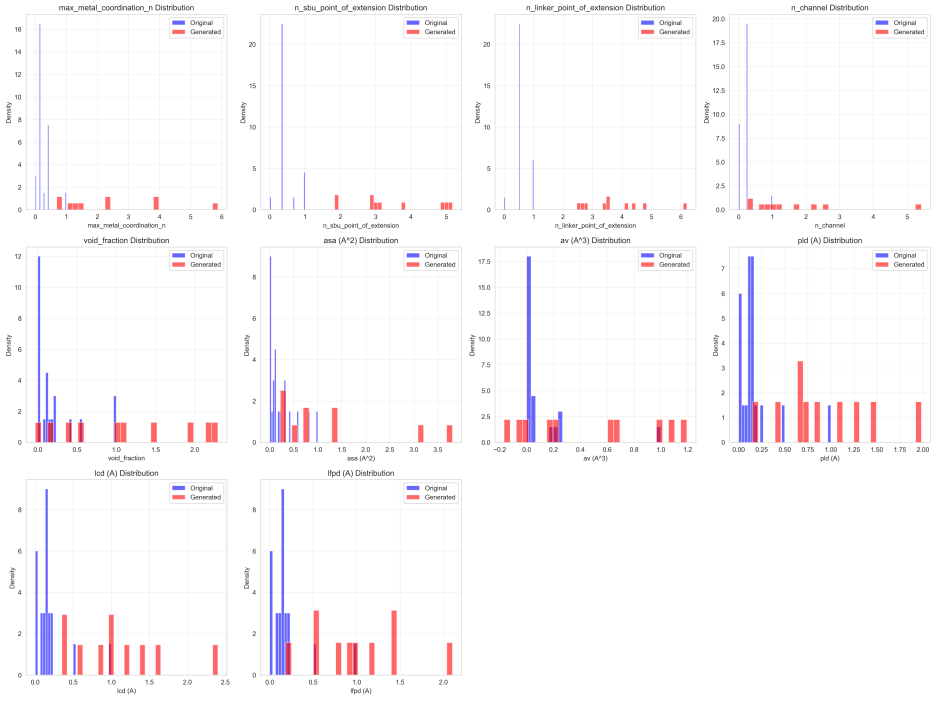


Figure 9: Comparison of feature distributions between training and generated MOFs across all descriptor features. Some features show similar distributions while others exhibit exploration of underrepresented regions.

7.6 Dimensionality Reduction Visualization

Principal component analysis of combined training and generated MOF features reveals the distribution of structures in reduced-dimensional space, as

shown in Figure 10. Generated nodes occupy regions both overlapping with and distinct from training data, indicating coverage of known regions while exploring new areas.

The PCA visualization shows that generated nodes are not confined to training data clusters, but rather explore gaps and extensions of training distributions. This behavior supports the scenario of sparse exploration, wherein generated nodes fill gaps in existing MOF databases.

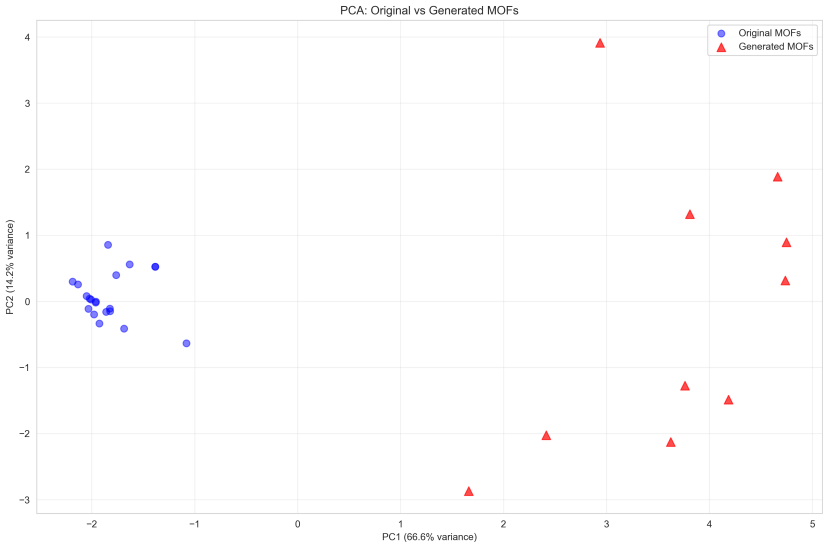


Figure 10: Principal component analysis visualization showing the distribution of training and generated MOFs in reduced-dimensional space. Generated nodes explore gaps and extensions of training distributions.

7.7 Feature Correlation Preservation

Comparison of feature correlation matrices between training and generated MOFs, shown in Figure 11, indicates that AGN preserves many domain relationships encoded in feature correlations. Strong correlations present in training data (e.g., between related pore size descriptors) are maintained in generated structures, suggesting that the model learns meaningful domain relationships rather than generating independent features.

However, some correlation differences emerge, potentially indicating discovery of new feature relationships or artifacts of the generation process. These differences warrant further investigation to distinguish between meaningful exploration and generation artifacts.

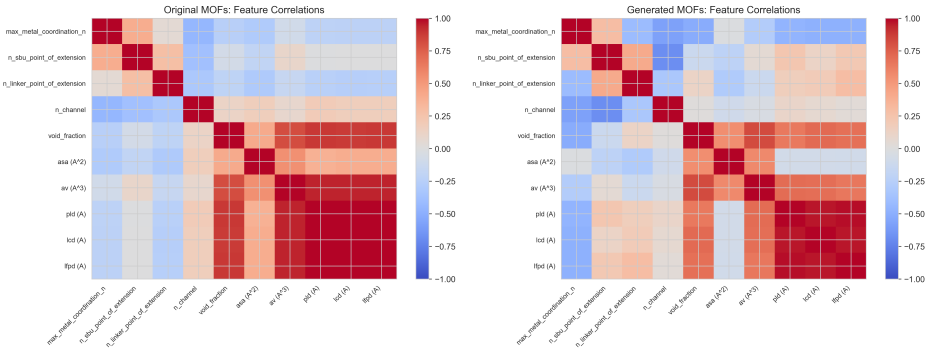


Figure 11: Feature correlation matrices for training and generated MOFs. The preservation of strong correlations indicates that AGN learns meaningful domain relationships.

7.8 Network Topology Preservation Before and After AGN Insertion

A critical requirement for AGN is that generated nodes integrate into the existing graph structure without disrupting global topology. To assess this, we compare network topology metrics before and after inserting generated nodes into the MOF similarity graph.

The original graph contains 1000 MOF nodes with edges encoding similarity relationships (similarity threshold $\tau = 0.5$). After inserting 10 generated nodes using the topology-preserving insertion procedure described in Section 4.4, the augmented graph contains 1010 nodes. Figure 12 shows side-by-side visualizations of the network before and after insertion, using consistent layout algorithms to enable fair comparison.

Visual inspection reveals that the network structure remains largely intact, with generated nodes (shown in red) integrating naturally into the existing topology. The connectivity backbone of the original network is preserved, with generated nodes connecting primarily to their nearest neighbors in feature space.

Table 1 presents quantitative topology metrics comparing the networks before and after insertion. Key observations include:

Stability of global properties: The number of connected components remains constant (2 components), and the giant component size increases appropriately from 999 to 1009 nodes, indicating that generated nodes integrate into the main connected component rather than forming isolated clusters. The average shortest path length increases slightly from 1.65 to 1.67, reflecting the addition of nodes while maintaining overall connectivity.

Preservation of local structure: The average clustering coefficient de-

creases minimally from 0.744 to 0.741, indicating that local clustering patterns are largely preserved. Assortativity remains stable (0.217 vs. 0.220), suggesting that degree-degree correlations are maintained.

Expected changes: Network density decreases slightly from 0.373 to 0.365, and average degree decreases from 372 to 369, both expected consequences of adding nodes with limited connectivity (top-k connections) relative to the highly connected original nodes. These changes are modest and reflect the sparse insertion strategy.

Figure 13 compares degree distributions before and after insertion. The distributions show substantial overlap, with the post-insertion distribution extending slightly toward lower degrees due to the sparser connectivity of generated nodes. The preservation of the overall degree distribution shape indicates that AGN insertion maintains the network's fundamental connectivity structure.

These results demonstrate that AGN successfully preserves global topology while integrating new nodes. The topology-preserving insertion procedure ensures that generated nodes enhance network coverage without disrupting existing connectivity patterns, supporting the scenario of sparse graph completion.

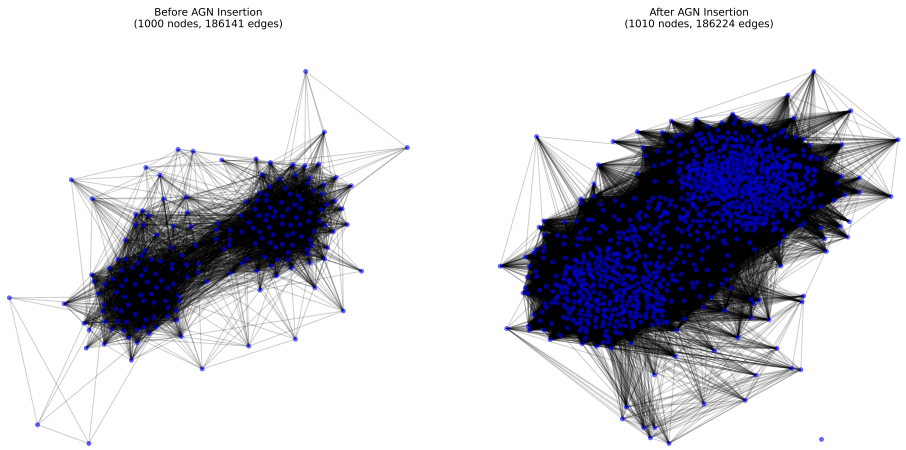


Figure 12: Network visualizations before and after AGN node insertion. The left panel shows the original network (1000 nodes, blue), and the right panel shows the augmented network with generated nodes highlighted in red. The consistent layout enables direct comparison of network structure.

Metric	Before	After
Num Nodes	1000.0000	1010.0000
Num Edges	186141.0000	186224.0000
Density	0.3727	0.3655
Avg Degree	372.2820	368.7604
Median Degree	385.0000	382.0000
Num Components	2.0000	2.0000
Giant Component Size	999.0000	1009.0000
Avg Component Size	500.0000	505.0000
Avg Clustering	0.7442	0.7410
Avg Shortest Path Length	1.6520	1.6666
Assortativity	0.2174	0.2197

Table 1: Network topology metrics before and after AGN node insertion.

8 Discussion

The results demonstrate that AGN successfully generates novel nodes that maintain domain plausibility while exploring previously unrepresented regions of design space. The conditional architecture enables targeted generation for specific constraint combinations, supporting scenario-driven exploration relevant to domain-specific workflows.

The balance between novelty and realism achieved by AGN is particularly noteworthy. Generated nodes exhibit significant distances from training data, confirming novelty, while maintaining feature distributions and correlations consistent with known structures. This balance suggests that AGN learns underlying patterns in domain relationships rather than simply memorizing training examples.

Critically, the topology analysis demonstrates that AGN preserves global network structure while integrating new nodes. The topology-preserving insertion procedure ensures that generated nodes enhance network coverage without disrupting existing connectivity patterns. Quantitative metrics show minimal changes in clustering, assortativity, and shortest path lengths, while the number of connected components remains stable. This preservation of topology is essential for maintaining the semantic meaning of graph-structured representations, ensuring that generated nodes integrate naturally into existing analysis workflows.

The descriptor-level representation enables efficient generation suitable for screening workflows, where detailed structural information is not immediately required. Generated nodes can serve as candidates for property predic-

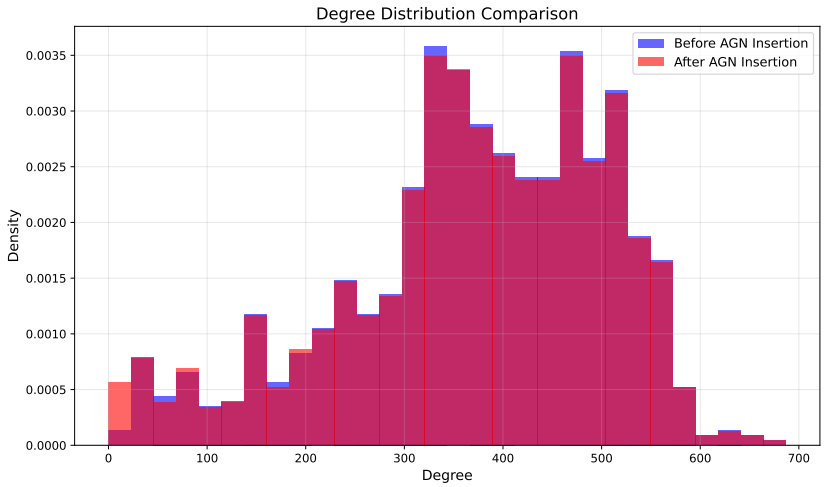


Figure 13: Degree distribution comparison before and after AGN node insertion. The substantial overlap indicates preservation of network connectivity structure, with slight extension toward lower degrees due to sparser connectivity of generated nodes.

tion, with subsequent structure optimization steps applied only to promising candidates.

However, several limitations must be acknowledged. First, the descriptor-level representation does not guarantee that generated feature combinations correspond to physically realizable structures. Second, domain validity checks provide necessary but not sufficient conditions for domain plausibility. Third, the fixed graph topology assumption limits exploration to regions near existing structures, though the topology-preserving insertion procedure mitigates disruption to global structure.

The evaluation framework establishes multiple complementary metrics for assessing generation quality, but no single metric fully captures the utility of generated structures for downstream applications. Integration with property prediction models and structure optimization algorithms would provide more direct assessment of practical utility.

The conditional generation mechanism, while enabling targeted exploration, relies on constraint combinations from training data. Extension to novel combinations would require modifications to the condition representation, potentially incorporating learned embeddings or domain-specific descriptors.

Most importantly, AGN represents a general framework applicable beyond the MOF domain demonstrated here. The framework can be instantiated for any graph-structured domain where nodes are characterized by

feature vectors and edges encode relationships. Future work will explore AGN's applicability to biological networks, social graphs, knowledge graphs, and other network domains.

9 Conclusion

This work presents AGN, a general graph-based generative framework for generating new nodes and optionally new edges in complex networks. AGN is not limited to any specific domain; rather, it provides a general paradigm applicable to any graph-structured representation. The framework operates at the descriptor level, generating nodes characterized by feature vectors and adjacency relationships, enabling integration with existing graph-based workflows.

As a demonstration of AGN's general applicability, we instantiate the framework for MOF node generation within graph-structured MOF representations. Comprehensive evaluation demonstrates that AGN generates novel MOF nodes that maintain realistic feature distributions while exploring previously unrepresented regions of MOF space. The conditional architecture supports targeted generation for specific metal-linker combinations, relevant to experimental synthesis workflows.

The work establishes AGN as a general paradigm for graph-aware generative modeling, with demonstrated applicability to materials informatics and extensibility to other network domains. The topology-preserving insertion procedure ensures that AGN-generated nodes integrate naturally into existing graph structures, maintaining global topology while enabling sparse graph completion.

Future directions include: (1) extension to edge generation for dynamic graph construction, enabling AGN to generate both nodes and edges simultaneously; (2) incorporation of property-based conditions, enabling targeted generation of nodes with specific properties; (3) integration with structure prediction algorithms to validate generated descriptors; and (4) exploration of AGN's applicability to biological networks, social graphs, knowledge graphs, and other network domains where sparse graph completion is needed.

AGN's general framework provides a foundation for addressing the fundamental challenge of sparse graph completion across diverse scientific and engineering domains. The demonstrated preservation of network topology, combined with the ability to generate novel nodes that explore previously unrepresented regions of design space, positions AGN as a versatile tool for computational exploration in graph-structured domains. The MOF instantiation serves as a proof-of-concept demonstrating AGN's capabilities, with

clear pathways for extension to other domains where entities are represented as nodes in similarity or interaction graphs.

Algorithms

This section presents two algorithms: Algorithm 1 describes the training procedure for the AGN generator and critic networks (see Section 4.2 for architecture details), and Algorithm 2 describes the conditional node generation and graph integration procedure.

Algorithm 1 AGN Training Procedure

Require: Training graph $G_{\text{train}} = (V_{\text{train}}, E_{\text{train}})$; node features $\mathbf{F}_{\text{train}} \in \mathbb{R}^{|V_{\text{train}}| \times d}$; adjacency representation $\mathbf{A}_{\text{train}} \in \mathbb{R}^{|V_{\text{train}}| \times D}$; condition vectors $\mathbf{C}_{\text{train}}$; generator G; critic D; hyperparameters λ_{GP}, λ_{adj}, λ_{feat}, n_{critic}, learning rates α_G, α_D

Ensure: Trained generator G and critic D

Initialize generator G and critic D with random weights
for epoch = 1 to max_epochs **do**
  for each batch $(\mathbf{F}_b, \mathbf{A}_b, \mathbf{C}_b)$ in training data **do**
    for $t = 1$ to n_{critic} **do**
      Sample latent noise: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, 0.25\mathbf{I})$
      Generate fake nodes: $(\mathbf{F}_{\text{fake}}, \mathbf{A}_{\text{fake}}) \leftarrow G(\mathbf{z}, \mathbf{C}_b)$
      Sample interpolation: $\hat{\mathbf{x}} \leftarrow \epsilon \mathbf{x}_{\text{real}} + (1 - \epsilon)\mathbf{x}_{\text{fake}}$ where $\epsilon \sim \mathcal{U}(0, 1)$
      Compute critic loss: $\mathcal{L}_C \leftarrow \mathbb{E}[D(\mathbf{x}_{\text{fake}})] - \mathbb{E}[D(\mathbf{x}_{\text{real}})] + \lambda_{\text{GP}}\mathbb{E}[(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1)^2]$
      Update critic: $\theta_D \leftarrow \theta_D - \alpha_D \nabla_{\theta_D} \mathcal{L}_C$
    end for
    Sample latent noise: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, 0.25\mathbf{I})$
    Generate fake nodes: $(\mathbf{F}_{\text{fake}}, \mathbf{A}_{\text{fake}}) \leftarrow G(\mathbf{z}, \mathbf{C}_b)$
    Compute generator loss: $\mathcal{L}_G \leftarrow -\mathbb{E}[D(\mathbf{x}_{\text{fake}})] + \lambda_{\text{adj}}\mathcal{L}_{\text{adj}} + \lambda_{\text{feat}}\mathcal{L}_{\text{feat}}$
    Update generator: $\theta_G \leftarrow \theta_G - \alpha_G \nabla_{\theta_G} \mathcal{L}_G$
  end for
  if early stopping criterion met **then**
    Break
  end if
end for
return Trained G and D

Algorithm 2 presents the complete procedure for generating new synthetic nodes using AGN with specified conditions. The algorithm operates

on a graph $G = (V, E)$ where nodes $v \in V$ are characterized by feature vectors $\mathbf{f}_v \in \mathbb{R}^d$ and edges encode relationships. The procedure generates new nodes conditioned on domain-specific constraints and integrates them into the existing graph structure while preserving topology.

Algorithm 1 describes the adversarial training procedure for AGN. The generator G and critic D are trained alternately using the Wasserstein GAN objective with gradient penalty. The generator learns to produce realistic node features and adjacency representations conditioned on domain-specific constraints, while the critic learns to distinguish real from generated nodes. The training incorporates reconstruction losses (cosine similarity) to encourage generated nodes to be similar to real nodes while maintaining diversity through adversarial training.

Algorithm 2 proceeds in three main phases: (1) *conditional generation*, where latent noise vectors are sampled and combined with encoded conditions to generate node features and adjacency representations through the trained generator G (architecture described in Section 4.2); (2) *graph augmentation*, where generated nodes are added to the original graph; and (3) *topology-preserving insertion*, where edges are created between generated nodes and existing nodes based on feature similarity, ensuring that generated nodes integrate naturally into the network structure. The hyperparameters k (number of nearest neighbors) and τ (similarity threshold) control the sparsity and connectivity of inserted nodes, enabling fine-tuning of the insertion strategy to balance exploration and topology preservation.

Acknowledgements

The authors acknowledge computational resources and data sources that enabled this work. Specific acknowledgements will be added during the publication process.

References

[1] Yaghi, O. M. et al. Reticular synthesis and the design of new materials. *Nature* **423**, 705–714 (2003).

[2] Colón, Y. J. et al. The computational design of metal-organic frameworks. *Current Opinion in Chemical Engineering* **7**, 75–81 (2015).

[3] Wilmer, C. E. et al. Large-scale screening of hypothetical metal-organic frameworks. *Nature Chemistry* **4**, 83–89 (2012).

[4] Xie, T. & Grossman, J. C. Crystal graph convolutional neural networks for an accurate and interpretable prediction of material properties. *Physical Review Letters* **120**, 145301 (2018).

[5] Similarity graph representations for materials networks. See [4, 10] for related approaches.

[6] Gómez-Bombarelli, R. et al. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science* **4**, 268–276 (2018).

[7] Zhou, Z. et al. Optimizing molecules using efficient queries from property evaluations. *Nature Machine Intelligence* **1**, 1–8 (2019).

[8] Wilmer, C. E. et al. Large-scale screening of hypothetical metal-organic frameworks. *Nature Chemistry* **4**, 83–89 (2012).

[9] Anderson, G. & Gómez-Gualdrón, D. A. Increasing topological diversity during computational MOF discovery: how much is enough? *Chemistry of Materials* **31**, 7153–7163 (2019).

[10] Boyd, P. G. & Woo, T. K. A generalized method for constructing hypothetical nanoporous materials of any net topology from graph theory. *CrystEngComm* **18**, 3777–3792 (2016).

[11] Kusner, M. J. et al. Grammar variational autoencoder. *International Conference on Machine Learning* (2017).

[12] Ragoza, M. et al. Generating 3D molecular structures conditional on a 3D electron density map. *Chemical Science* **13**, 13598–13610 (2022).

[13] Kingma, D. P. & Welling, M. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).

[14] Goodfellow, I. et al. Generative adversarial nets. *Advances in Neural Information Processing Systems* **27** (2014).

[15] Mirza, M. & Osindero, S. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784* (2014).

[16] Sanchez-Lengeling, B. et al. Inverse molecular design using machine learning: generative models for matter engineering. *Science* **361**, 360–365 (2018).

[17] Gulrajani, I. et al. Improved training of wasserstein GANs. *Advances in Neural Information Processing Systems* **30** (2017).

[18] Miyato, T. et al. Spectral normalization for generative adversarial networks. *arXiv preprint arXiv:1802.05957* (2018).

[19] Barabási, A.-L. & Oltvai, Z. N. Network biology: understanding the cell's functional organization. *Nature Reviews Genetics* **5**, 101–113 (2004).

[20] Newman, M. E. The structure and function of complex networks. *SIAM Review* **45**, 167–256 (2003).

[21] Bordes, A. et al. Translating embeddings for modeling multi-relational data. *Advances in Neural Information Processing Systems* **26** (2013).

[22] Leskovec, J. & Faloutsos, C. Sampling from large graphs. *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2006).

Algorithm 2 AGN Conditional Node Generation and Graph Integration

Require: Trained generator $G : \mathbb{R}^z \times \mathbb{R}^c \to \mathbb{R}^d \times \mathbb{R}^D$; original graph $G_{\text{orig}} = (V_{\text{orig}}, E_{\text{orig}})$; original node features $\mathbf{F}_{\text{orig}} \in \mathbb{R}^{|V_{\text{orig}}| \times d}$; condition encoder Enc; number of samples n; hyperparameters k, τ; training statistics $\mathbf{f}_{\min}$, $\mathbf{f}_{\max}$, $\mu_{\mathbf{A}}$, $\sigma_{\mathbf{A}}$

Ensure: Augmented graph $G_{\text{aug}} = (V_{\text{aug}}, E_{\text{aug}})$ with generated nodes

Initialize empty list $\mathbf{F}_{\text{gen}} \leftarrow []$
Initialize empty list $\mathbf{A}_{\text{gen}} \leftarrow []$
for each condition vector $\mathbf{c}$ **do**
  Encode condition: $\mathbf{c}^{\text{enc}} \leftarrow \text{Enc}(\mathbf{c})$
  for $i = 1$ to n **do**
    Sample latent noise: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, 0.25\mathbf{I})$
    Generate normalized node: $(\mathbf{f}_{\text{gen}}^{\text{norm}}, \mathbf{a}_{\text{gen}}^{\text{std}}) \leftarrow G(\mathbf{z}, \mathbf{c}^{\text{enc}})$
    Denormalize features: $\mathbf{f}_{\text{gen}} \leftarrow \mathbf{f}_{\text{gen}}^{\text{norm}} \odot (\mathbf{f}_{\max} - \mathbf{f}_{\min}) + \mathbf{f}_{\min}$
    Denormalize adjacency: $\mathbf{a}_{\text{gen}} \leftarrow \mathbf{a}_{\text{gen}}^{\text{std}} \odot \sigma_{\mathbf{A}} + \mu_{\mathbf{A}}$
    Append to lists: $\mathbf{F}_{\text{gen}} \leftarrow \mathbf{F}_{\text{gen}} \cup \{\mathbf{f}_{\text{gen}}\}$, $\mathbf{A}_{\text{gen}} \leftarrow \mathbf{A}_{\text{gen}} \cup \{\mathbf{a}_{\text{gen}}\}$
  end for
end for
Initialize augmented graph: $G_{\text{aug}} \leftarrow G_{\text{orig}}$
for each generated feature vector $\mathbf{f}_{\text{gen}} \in \mathbf{F}_{\text{gen}}$ **do**
  Create new node: $v_{\text{new}} \leftarrow \text{AddNode}(G_{\text{aug}})$
  Compute similarities: $\mathbf{s} \leftarrow \text{CosineSimilarity}(\mathbf{f}_{\text{gen}}, \mathbf{F}_{\text{orig}})$
  Find top-k neighbors: $\mathcal{N}_k \leftarrow \text{TopK}(\mathbf{s}, k)$
  for each neighbor $v_j \in \mathcal{N}_k$ **do**
    if $\mathbf{s}[j] \geq \tau$ **then**
      Add edge: $E_{\text{aug}} \leftarrow E_{\text{aug}} \cup \{(v_{\text{new}}, v_j)\}$ with weight $\mathbf{s}[j]$
    end if
  end for
end for
Optionally connect generated nodes: For each pair $(\mathbf{f}_i, \mathbf{f}_j)$ in $\mathbf{F}_{\text{gen}}$, if $\text{CosineSimilarity}(\mathbf{f}_i, \mathbf{f}_j) \geq \tau$, add edge (v_i, v_j)
return G_{aug}